Naming and Structure Guidelines for FastAPI Projects

---

1. Python Project Structure

A clean and scalable FastAPI project typically follows this structure:

```
my_project/
│
├── app/
│   ├── main.py                 # Application entry point
│   ├── core/
│   │   ├── logger.py           # Central logging
│   │   ├── config.py           # Configs like DB URL, ENV variables
│   │   └── security.py         # Authentication, JWT, etc.
│   │
│   ├── models/
│   │   ├── employee.py          # ORM models (SQLAlchemy)
│   │   └── phone_number.py
│   │
│   ├── schemas/
│   │   ├── employee_schema.py   # Pydantic models
│   │   └── phone_schema.py
│   │
│   ├── routers/
│   │   ├── employee_router.py   # Routes for employees
│   │   ├── phone_router.py
│   │   └── salary_router.py
│   │
│   ├── services/
│   │   └── employee_service.py  # Business logic
│   │
│   └── database/
│       └── database.py          # DB connection, async engine, sessions
│
├── tests/
│   ├── test_employee.py
│   └── test_phone.py
│
├── requirements.txt
└── .env
```

---

2. Naming Conventions

2.1 Classes

- Use **PascalCase** (CapWords) for classes.
- Names should be **descriptive** of the entity or functionality.

Examples:

Entity	Class Name
Employee ORM model	`Employee`
Phone Number model	`PhoneNumber`
Pydantic schema	`EmployeeCreate`, `EmployeeUpdate`, `PhoneNumberRequest`
Service class	`EmployeeService`

2.2 Variables

- Use **snake_case** for variables and function names.
- Names should be **clear and meaningful**.
- Avoid single letters (except common loop variables like `i` or `j`).

Examples:

```
emp_id: int
first_name: str
last_name: str
date_of_birth: str
salary: float
status: str
phone_number: str
connection: AsyncConnection
```

2.3 Files

- Use **snake_case** for Python files.
- The file name should indicate its content or module responsibility.

Examples:

File Type	File Name
Router	`employee_router.py`
Model	`employee.py`
Pydantic Schema	`employee_schema.py`
Service/Business Logic	`employee_service.py`
DB	`database.py`
Logger	`logger.py`

2.4 Folder Names

- Use lowercase and snake_case.
- Group related modules.

Examples:

- `routers/` → all API routes
- `models/` → all ORM models
- `schemas/` → Pydantic models
- `services/` → business logic functions
- `core/` → config, logger, security, and central utilities
- `database/` → engine, async session, connection handling

---

3. FastAPI Endpoints Naming Conventions

Endpoints should be **RESTful**, **predictable**, and **resource-oriented**.

3.1 HTTP Methods Mapping

Operation	HTTP Method	Example Endpoint
Create Employee	POST	`/employees/`
Get Employee by ID	GET	`/employees/{employee_id}`
Get Employees by Name	GET	`/employees/by_name/{name}`
Update Employee Salary	PUT	`/employees/{employee_id}/salary`
Delete Employee	DELETE	`/employees/{employee_id}`
Add Phone Number	POST	`/employees/{employee_id}/phones`
Delete Phone Number	DELETE	`/employees/{employee_id}/phones/{phone_id}`

3.2 Naming Best Practices

- Use **plural nouns** for resources: `/employees`, `/phones`
- Use **path parameters** for specific resources: `/employees/{employee_id}`
- Use **nested resources** for related entities: `/employees/{employee_id}/phones`
- Use **action-specific paths** only when necessary (e.g., `/employees/{employee_id}/salary` for salary update)

---

4. Logging Best Practices

4.1 Central Logger

- Create one logger in `core/logger.py`.
- Configure:
  - Rotating file logs
  - Console output
  - Standard format: timestamp, logger name, level, message

4.2 Logging Levels

- **DEBUG** → development debug information
- **INFO** → normal operations (e.g., employee created)
- **WARNING** → non-critical issues
- **ERROR** → unexpected errors
- **EXCEPTION** → includes stack trace

4.3 Example Usage

```
from core.logger import logger

logger.info("Employee record created")
logger.warning("Employee already exists")
logger.error("Database connection failed")
logger.exception("Exception while deleting employee")
```

---

5. Async/Await Guidelines

- Use **`async def`** for route handlers and DB operations.
- Use **`async with`** for async sessions or connections.
- Ensure **await** for all asynchronous calls (`await connection.execute(...)`)

---

6. Pydantic Schemas

- Separate request and response schemas.
- Use **suffixes** for clarity:
  - `Create` → for POST requests
  - `Update` → for PUT/PATCH
  - `Response` → for response models

Example:

```
class EmployeeCreate(BaseModel):
    first_name: str
```

```
    last_name: str
    date_of_birth: str
    salary: float

class EmployeeUpdateSalary(BaseModel):
    salary: float

class EmployeeResponse(BaseModel):
    emp_id: int
    first_name: str
    last_name: str
```

---

7. SQLAlchemy ORM Models

- Class names: PascalCase
- Table names: lowercase, plural (optional)
- Columns: snake_case
- Use **`__tablename__`** for table name

Example:

```
from sqlalchemy import Column, Integer, String, Float, Date
from sqlalchemy.orm import declarative_base

Base = declarative_base()

class Employee(Base):
    __tablename__ = "employee_details"

    emp_id = Column(Integer, primary_key=True, autoincrement=True)
    first_name = Column(String(50), nullable=False)
    last_name = Column(String(50), nullable=False)
    date_of_birth = Column(Date, nullable=False)
    salary = Column(Float, nullable=False)
    status = Column(String(10), nullable=False)
```

---

8. Recommended Python Style

- Follow **PEP8**: max 79 characters per line, 4-space indent, lowercase module names.
- Use **docstrings** for modules, classes, and functions.
- Keep **one responsibility per module**.
- Use **type hints** for all functions.

---

9. Example Endpoint Implementation (Employee Salary Update)

```
from fastapi import APIRouter, HTTPException
from sqlalchemy import text
from sqlalchemy.exc import SQLAlchemyError
from db.database import engine
from core.logger import logger

router = APIRouter()

@router.put("/employees/{employee_id}/salary")
async def update_salary(employee_id: int, new_salary: float):
    try:
        async with engine.connect() as connection:
            emp_exists = await connection.execute(
                text("SELECT 1 FROM employee_details WHERE emp_id = 
:employee_id"),
                {"employee_id": employee_id}
            )
            if not emp_exists.fetchone():
                logger.warning(f"Employee {employee_id} not found")
                raise HTTPException(status_code=404, detail="Employee not 
found")

            await connection.execute(
                text("UPDATE employee_details SET salary = :salary WHERE 
emp_id = :employee_id"),
                {"salary": new_salary, "employee_id": employee_id}
            )
            await connection.commit()
            logger.info(f"Updated salary for employee {employee_id} to 
{new_salary}")

        return {"status": "success", "message": "Salary updated 
successfully"}

    except Exception as e:
        logger.exception(f"Error updating salary for employee {employee_id}")
        raise HTTPException(status_code=500, detail="Internal server error")
```

This documentation provides **complete professional guidance** for:

- File and folder structure
- Classes, variables, and functions naming
- Logging standards and usage
- Pydantic schemas
- SQLAlchemy ORM naming
- FastAPI endpoint conventions